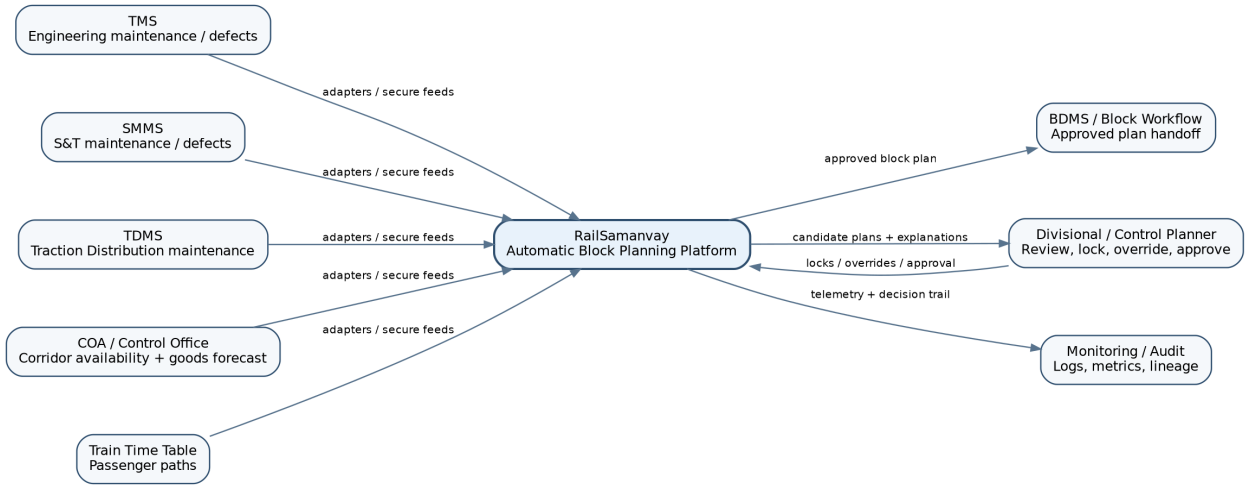
SIH26027

Detailed Technical Architecture

AI-Powered Automatic Block Planning to Maximize Asset Availability for Train Operations on Indian Railways

Architecture objective

Design a production-oriented decision-support platform that ingests maintenance demand from Engineering, Signal & Telecommunication and Traction Distribution, combines it with corridor availability and train-path demand, and produces explainable weekly/monthly maintenance block plans. The architecture deliberately separates predictive ML from deterministic constraint-based scheduling.



Scope: technical architecture only - components, data contracts, optimization services, APIs, storage, deployment, security, observability, resilience and scaling.

Contents

1. Architecture Scope and Design Principles

The problem statement calls for integration of TMS, SMMS and TDMS maintenance/defect data with block/corridor availability, passenger Train Time Table information and goods-train forecasts from the Control Office, followed by AI/ML based prioritization and optimized weekly/monthly block planning. The architecture below treats those source systems as external authoritative systems and introduces a separate planning platform rather than replacing them.

The production design should be a decision-support and planning system. It may recommend, compare and re-optimize candidate block plans, but sanctioning and operational execution remain under authorized Railway workflows and personnel.

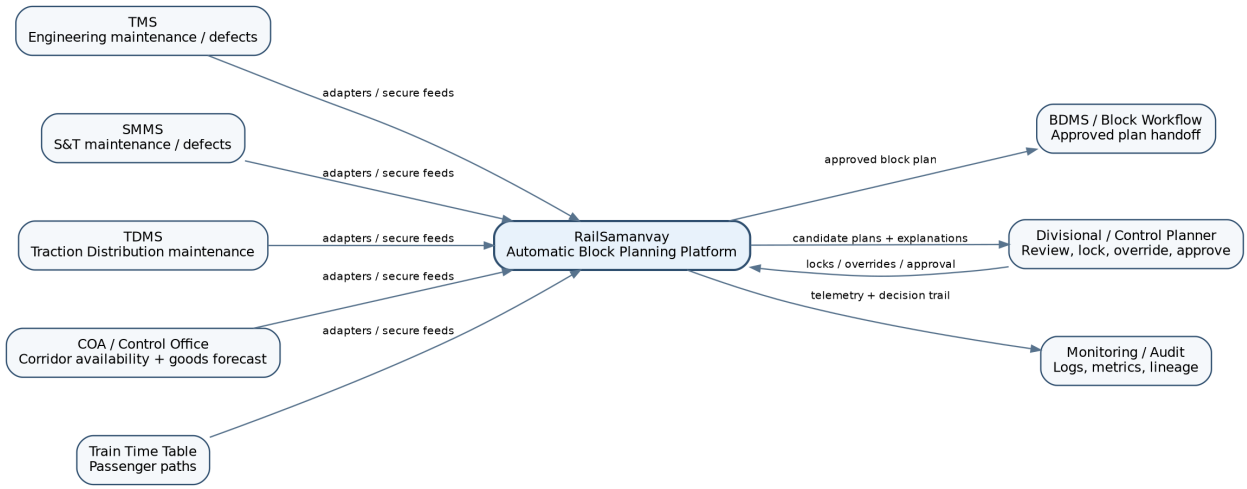
Core design principles

- Safety constraints before optimization: hard operating constraints cannot be violated to improve a score.
- Prediction is not scheduling: ML services estimate urgency, duration and uncertainty; a constraint solver selects feasible times.
- Human in the loop: planner locks, overrides and rejections become explicit constraints in subsequent runs.
- Snapshot reproducibility: every planning run operates on an immutable versioned input snapshot.
- Explainability: each scheduled or unscheduled job receives machine-readable reason codes and factor contributions.
- Adapter isolation: source-specific logic is confined to connectors; core planning works on a canonical schema.
- Graceful degradation: if ML is unavailable, rule-based scores and conservative duration estimates allow planning to continue.
- Audit by construction: source lineage, configuration, model version, solver version and user overrides are stored with every result.

Recommended architectural style

A modular service-oriented backend with a single transactional PostgreSQL/PostGIS core is appropriate for SIH and an initial pilot. Avoid premature microservices. Separate services only where isolation matters: ingestion, optimization workers and optional ML inference. This gives reliability without adding distributed-system complexity.

2. System Context

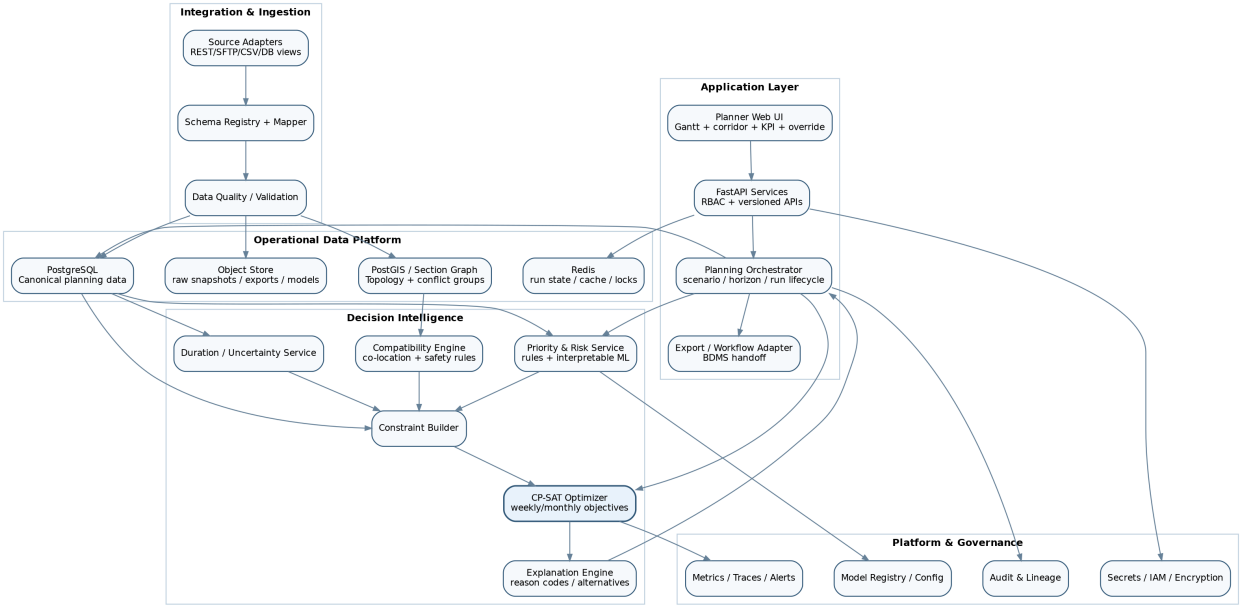


The platform sits between Railway operational data sources and the planner. In the prototype, adapters can read controlled CSV/JSON snapshots. In a sanctioned deployment, the same contracts can be backed by secure APIs, database views, file exchanges or event feeds approved by the respective system owners.

External interfaces

System / actor	Data consumed or produced	Architecture treatment
TMS	Engineering defects, inspections, overdue maintenance, section/asset identifiers	Read-only source adapter; normalize into maintenance_job and asset entities
SMMS	S&T; defects, planned work, disconnection requirements	Read-only source adapter; map signalling work and dependencies
TDMS	Traction Distribution defects/planned maintenance/isolation needs	Read-only source adapter; map TRD work and power-isolation constraints
COA / Control Office	Corridor availability, operational occupancy, goods-train forecast	Read/forecast adapter; converted into train/path occupancy and candidate windows
Train Time Table	Scheduled passenger paths	Timetable parser / canonical path occupancy
BDMS	Existing block request/sanction workflow	Outbound approved-plan adapter; exact write semantics must be validated
Planner / controller	Manual review, locks, overrides, approval	Web UI + authenticated planning APIs

3. Logical Component Architecture



The platform is divided into five logical areas: integration, operational data, decision intelligence, application services and platform governance. The critical path for planning is intentionally short: canonical snapshot -> feature/rule evaluation -> constraint construction -> solver -> explanation/KPIs -> planner review.

Component responsibilities

Component	Responsibility	Key input/output
Source Adapter Framework	Pull/push source data, checkpointing, deduplication, source-specific mapping	Raw source records -> normalized staging records

Component	Responsibility	Key input/output
Schema Mapper & Data Quality	Required-field validation, enum mapping, date/time normalization, referential checks	Valid records -> canonical entities; invalid -> quarantine
Planning Snapshot Service	Freezes versioned inputs for a run	snapshot_id referencing jobs, paths, resources, rules, config
Priority & Risk Service	Calculates urgency and failure-risk contribution	job features -> score + explanation vector
Duration Service	Predicts/derives duration bounds and buffers	job + historical actuals -> expected/p80/p95 duration
Compatibility Engine	Determines whether jobs may co-exist in one block	job pairs/groups -> compatibility/conflict reason
Constraint Builder	Converts domain rules into solver variables/constraints	snapshot + rules -> CP-SAT model
Optimizer Worker	Solves bounded weekly/monthly problem	model -> solution/status/gap/runtime
Explanation Engine	Produces reason codes, alternatives and KPI deltas	solver result + features -> explainable plan
Planning Orchestrator	Controls lifecycle, retries, cancellation, re-planning	scenario/run requests -> durable run state
Planner API	Versioned REST APIs, RBAC, optimistic concurrency	UI requests -> domain operations
Audit & Lineage	Immutable decision trail	actor, config, source hashes, model/solver versions, overrides

4. Source Integration Layer

The integration layer must protect the planning core from source-system differences. Each connector implements the same adapter contract and emits canonical staging events/records. In a hackathon prototype, a FileAdapter is enough; in a pilot, add API/DB adapters without changing downstream services.

Adapter contract

```
interface SourceAdapter: healthcheck() -> SourceHealth extract(cursor, window) -> RawBatch
normalize(raw_record) -> StagingRecord checkpoint(batch_id, cursor) -> void source_metadata() -> {system,
schema_version, timezone}
```

Required behaviors

- Idempotency: identical source records must not create duplicate jobs.
- Incremental checkpoints: adapters should support changed-since timestamps or source cursors where available.
- Source lineage: every canonical row stores source_system, source_record_id, ingestion_batch_id and source_updated_at.
- Timezone normalization: internal timestamps stored in UTC with original Railway/local timezone context retained; planner presentation uses the configured division timezone.
- Schema versioning: source mapping changes are explicit, testable and reversible.
- Quarantine: malformed records never silently enter optimization; they are held with validation codes.

Recommended ingestion modes

Mode	Use	Notes
Scheduled batch	TMS/SMMS/TDMS snapshots, timetable refresh	Simple, auditable and enough for weekly/monthly planning

Mode	Use	Notes
Near-real-time poll	Urgent defects or changed corridor availability	Optional; 5-15 minute cadence can be added in pilot
Event feed	Future production integration	Only if source systems provide reliable events; not required for SIH
Manual import	Prototype/offline fallback	CSV/JSON with schema validation and preview before commit

5. Canonical Railway Planning Data Model

The canonical schema is the architectural center. The optimizer should never depend directly on TMS/SMMS/TDMS field names. All services work with stable internal identifiers and explicit relationships.

Core entities

Entity	Selected fields	Purpose
maintenance_job	job_id, dept, asset_id, section_id, work_type, severity, due_at, earliest_start, latest_finish, duration_min/max, isolation_type, crew_type, status	Atomic schedulable demand
asset	asset_id, type, dept, section_id, criticality_class, failure_history_ref	Physical infrastructure being maintained
section	section_id, from_node, to_node, line, direction, electrification, conflict_group	Scheduling/topology unit
path_occupancy	path_id, train_class, section_id, enter_at, exit_at, priority_weight, confidence	Passenger/goods corridor use
candidate_window	window_id, section_id, start_at, end_at, allowed_work_types, source	Feasible placement supply
resource	resource_id/type, home_location, shift, capacity, availability	Crew/machine/specialist constraint
job_dependency	predecessor_job_id, successor_job_id, min_gap, dependency_type	Ordering constraint
job_compatibility	job_a, job_b, compatible, reason_code, required_shared_conditions	Co-location constraint
planning_snapshot	snapshot_id, horizon, source_versions, config_version, created_at	Immutable run input
planning_run	run_id, snapshot_id, solver_config, state, started_at, completed_at	Execution lifecycle
block_plan	plan_id, run_id, version, approval_state, KPI bundle	Candidate/approved output
block_assignment	plan_id, block_id, job_id, start/end, section, reason_codes	Concrete schedule result
user_override	run/plan id, object id, action, old/new value, user, timestamp, justification	Human decision audit

Identifier rule

Use platform-generated UUIDs internally and maintain a separate source_identity table for source-specific keys. This prevents collisions and allows the same physical asset to be mapped across multiple systems after data stewardship confirms the relationship.

6. Ingestion, Validation and Snapshot Pipeline

Every optimization run must be reproducible. Therefore the data pipeline creates a versioned planning snapshot rather than querying mutable source tables during a solve.

Validation stages

- Structural: required fields, parsable timestamps, permitted department/work-type values.
- Referential: job section/asset/resource references exist in canonical master data.
- Temporal: earliest_start <= latest_finish; duration > 0; path enter_at < exit_at.
- Operational plausibility: duration within configurable bounds for work type; isolation requirement known; section topology valid.
- Duplicate: same source job revision not ingested twice; conflicting revisions kept with lineage.
- Snapshot completeness: planner is warned if one source feed is stale or missing beyond its SLA.

Snapshot manifest example

```
{ "snapshot_id": "snap_2026W36_DLI_xxx", "horizon": {"start": "...", "end": "..."}, "sources": { "TMS": {"batch": "...", "as_of": "..."}, "SMMS": {"batch": "...", "as_of": "..."}, "TDMS": {"batch": "...", "as_of": "..."}, "COA": {"batch": "...", "as_of": "..."}, "TIMETABLE": {"version": "..."} }, "rule_set": "ruleset-v7", "model_versions": {"priority": "p3", "duration": "d5"} }
```

7. Corridor and Topology Model

Scheduling needs more than station names. A section graph represents the infrastructure and conflicts that make simultaneous blocks impossible. PostGIS may store geometry for visualization, while a graph representation handles adjacency and conflict relationships.

Graph model

- Nodes: stations, junctions, block boundaries or other planning boundaries.
- Edges: track sections with line/direction/electrification/capacity metadata.
- Conflict groups: edges or resources that cannot be blocked together even if identifiers differ.
- Isolation zones: TRD power-isolation regions that may cover multiple track sections.
- S&T; dependency zones: configurable regions for disconnections/interlocking dependencies.
- Path projection: each train path expands into time intervals over specific section edges.

Candidate window derivation

For each section and planning horizon, subtract protected train/path occupancy, fixed blocks, safety margins and planner-reserved periods from the time axis. The remaining intervals become candidate windows. The exact operational rules and margins must remain configuration, not hard-coded values, until validated by Railway domain experts.

8. Maintenance Priority and ML Services

The priority service produces an ordering signal; it does not grant permission to schedule. Hard constraints remain in the optimizer. The service should support both a deterministic rule mode and an ML-assisted mode.

Priority feature groups

- Safety/criticality class and defect severity.
- Overdue age relative to prescribed due date or inspection interval.
- Observed degradation/failure history where reliable labels are available.

- Asset operational importance and redundancy.
- Expected duration and restoration complexity.
- Opportunity score: whether an unusually suitable block window exists.
- Dependency impact: whether completing the job unblocks other maintenance.

Scoring contract

```
POST /internal/priority/score Input: job + asset context + feature timestamp Output: { "priority_score": 0.87, "priority_class": "P1", "factors": [ {"name":"criticality", "contribution":0.31}, {"name":"overdue", "contribution":0.18} ], "model_version":"priority-v3", "fallback_used": false }
```

Duration uncertainty

Use historical actuals if available to estimate expected and high-percentile duration. The optimizer should schedule a conservative bound (for example configurable p80/p90 or rule-based buffer), while the UI can display the expected duration separately. This reduces the chance that a statistically optimistic duration creates a fragile block plan.

9. Compatibility and Rule Engine

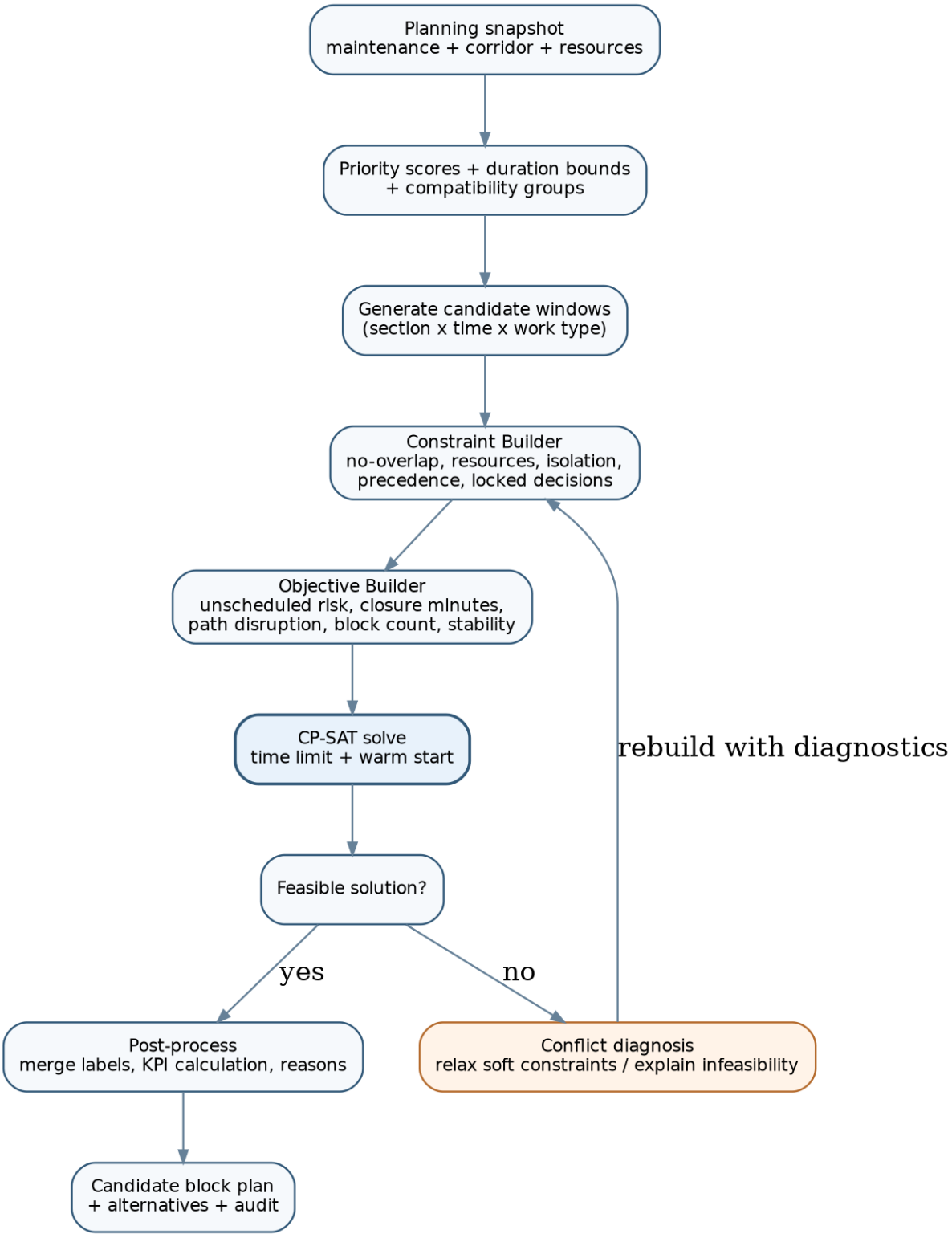
Multi-department coordination is the core value of the solution, but co-location must be explicit rather than assumed. A compatibility engine evaluates whether two or more jobs may share a block and what conditions must be true.

Rule categories

- Spatial: same section, overlapping isolation zone or physically compatible adjacent work areas.
- Temporal: duration fits within common block; setup/restoration sequence is feasible.
- Safety: two work types are not mutually exclusive; required protection can be established.
- Power: TRD isolation state required by each job is mutually consistent.
- Signalling: disconnection/interlocking requirements do not conflict.
- Resource: crew/machine requirements can be satisfied simultaneously.
- Precedence: ordering dependencies within a shared block are feasible.

Rules should be stored as versioned configuration, e.g. decision tables/JSON or a lightweight rules module. Every compatibility decision returns a reason code such as SAME_SECTION_OK, POWER_ISOLATION_CONFLICT or CREW_CAPACITY_CONFLICT. These reason codes feed both the solver and the planner explanation UI.

10. Constraint Optimization Engine



Google OR-Tools CP-SAT is a strong prototype choice because block planning is an interval/resource scheduling problem with boolean assignment decisions, optional intervals, no-overlap constraints, cumulative resource limits and multi-objective penalties.

Decision variables

- x[j,w] = 1 if maintenance job j is assigned to candidate window w.
- start[j], end[j] = scheduled interval for a selected job.
- block_used[w] = 1 when at least one maintenance assignment uses window w.

- shared_group[g,w] = optional variable for co-located compatible work.
- unscheduled[j] = 1 when job cannot/should not be placed in the horizon.
- change[j] = 1 when a re-plan moves a previously accepted assignment, used to penalize plan churn.

Hard constraint families

- Each job is assigned at most once; mandatory P0/P1 jobs can be configured as required unless infeasible.
- Assignment only to windows valid for job section, work type, isolation and earliest/latest bounds.
- No-overlap for mutually exclusive sections/conflict groups.
- Cumulative capacities for crews, machines and specialist resources.
- Precedence and minimum-gap constraints for dependent jobs.
- Protected train paths are excluded or represented as hard no-overlap intervals.
- Planner-locked assignments are fixed variables in re-optimization.
- Compatibility rules constrain which jobs may share one block.

Objective design

Use a priority-ordered or weighted objective. A safe default is lexicographic: first minimize weighted unscheduled safety/maintenance risk; then minimize operational closure/disruption; then improve consolidation and stability. If a weighted objective is used, weights must be configurable and normalized to avoid one metric accidentally dominating.

```
Minimize: W1 * SUM(priority_weight[j] * unscheduled[j]) + W2 * total_section_closure_minutes + W3 *
weighted_train_path_overlap_penalty + W4 * number_of_distinct_blocks + W5 *
resource_repositioning_or_idle_penalty + W6 * replanning_change_penalty
```

Solver controls

- Hard wall-clock time limit per run.
- Return best feasible solution and objective bound/gap if optimality is not proven.
- Warm-start from current approved/tentative plan during re-planning.
- Deterministic seed/config for reproducible benchmark scenarios.
- Partition very large horizons by corridor/time with coordination constraints at boundaries.
- Persist solver statistics: branches, conflicts, objective, bound, runtime and solution status.

11. Weekly and Monthly Planning Orchestration

Weekly and monthly planning should not be the same optimization run with a larger date range. They have different fidelity and decision intent.

Monthly planner

- Aggregates or buckets train/path demand at coarser granularity where appropriate.
- Reserves likely maintenance capacity by corridor/section and identifies backlog that cannot fit.
- Focuses on high-risk tasks, campaign work, machine planning and capacity shortfalls.
- Outputs tentative capacity reservations and preferred windows, not necessarily final minute-level sanctions.

Weekly planner

- Uses detailed path occupancy and exact candidate windows.
- Expands monthly reservations into executable candidate blocks.
- Applies current defects, changed resource availability and planner locks.
- Produces the concrete plan used for review and handoff.

Hierarchy

Monthly outputs become soft preferences/upper-level capacity constraints for weekly planning. Weekly planning may deviate when new critical defects or operational changes appear, but the deviation is recorded and surfaced as a KPI.

12. API and Application Architecture

The UI must never communicate directly with the database or solver. All operations flow through a versioned Planning API that applies RBAC, validation and concurrency control.

API modules

Module	Representative endpoints	Notes
Data readiness	GET /v1/sources/status, GET /v1/snapshots/{id}	Shows freshness and validation errors
Maintenance	GET /v1/jobs, GET /v1/jobs/{id}	Read-only canonical backlog
Scenarios	POST /v1/scenarios, POST /v1/scenarios/{id}/snapshot	Create reproducible planning scenario
Runs	POST /v1/runs, GET /v1/runs/{id}, POST /v1/runs/{id}/cancel	Asynchronous optimization lifecycle
Plans	GET /v1/plans/{id}, GET /v1/plans/{id}/kpis	Candidate result retrieval
Overrides	POST /v1/plans/{id}/locks, POST /v1/plans/{id}/reject	Creates audit entry and re-plan input
Re-plan	POST /v1/plans/{id}/reoptimize	Warm-starts from accepted plan
Export	POST /v1/plans/{id}/export	CSV/PDF/API handoff; production BDMS adapter later
Admin	GET/PUT /v1/rulesets, model/config status	Restricted role only

Concurrency

Plan resources carry a version/ETag. If two planners edit the same plan, stale writes are rejected with HTTP 409 and the UI asks the user to reload/merge. Locks are persistent business decisions, not transient browser locks.

13. Persistence, Caching and Run State

A relational database is the system of record because planning entities are highly relational and require transactions, lineage and audit queries. PostGIS extends it for corridor geometry; an object store holds raw snapshots and large exports; Redis is optional for ephemeral coordination.

Storage allocation

Store	Data	Why
PostgreSQL	Canonical master data, jobs, resources, snapshots metadata, runs, plans, overrides, audits	Transactions, joins, constraints, reproducible queries
PostGIS extension	Section geometry, spatial lookup, map rendering support	Spatial operations without a separate GIS database
Object store (S3/MinIO)	Raw source batches, immutable snapshot bundles, exports, trained model artifacts	Cheap immutable blob storage
Redis	Short-lived run progress, worker leases, response cache, distributed mutex where needed	Fast ephemeral state; never sole copy of approved plan
Metrics/log backend	Time-series metrics, structured logs, traces	Operational observability separated from domain data

Transaction boundary

Creating a snapshot or accepting a planner override should commit atomically. Solver execution itself runs outside the database transaction; it reads an immutable snapshot package and writes results as a new plan version only after successful solve/post-processing.

14. End-to-End Execution Sequences

A. Fresh weekly plan

- 1. Scheduler or planner requests a data refresh.
- 2. Adapters ingest source deltas; validation/quarantine completes.
- 3. Snapshot Service freezes jobs, paths, resources, rule/model versions.
- 4. Orchestrator creates planning_run(state=QUEUED) and submits worker job.
- 5. Priority/duration/compatibility services enrich the snapshot.
- 6. Constraint Builder creates CP-SAT model and solver runs under a time budget.
- 7. Result Post-Processor calculates closure minutes, backlog risk, affected paths and consolidation KPIs.
- 8. Explanation Engine attaches reason codes and alternatives.
- 9. Plan version 1 is persisted and run transitions to COMPLETED.
- 10. UI receives a progress event/poll result and loads the plan.

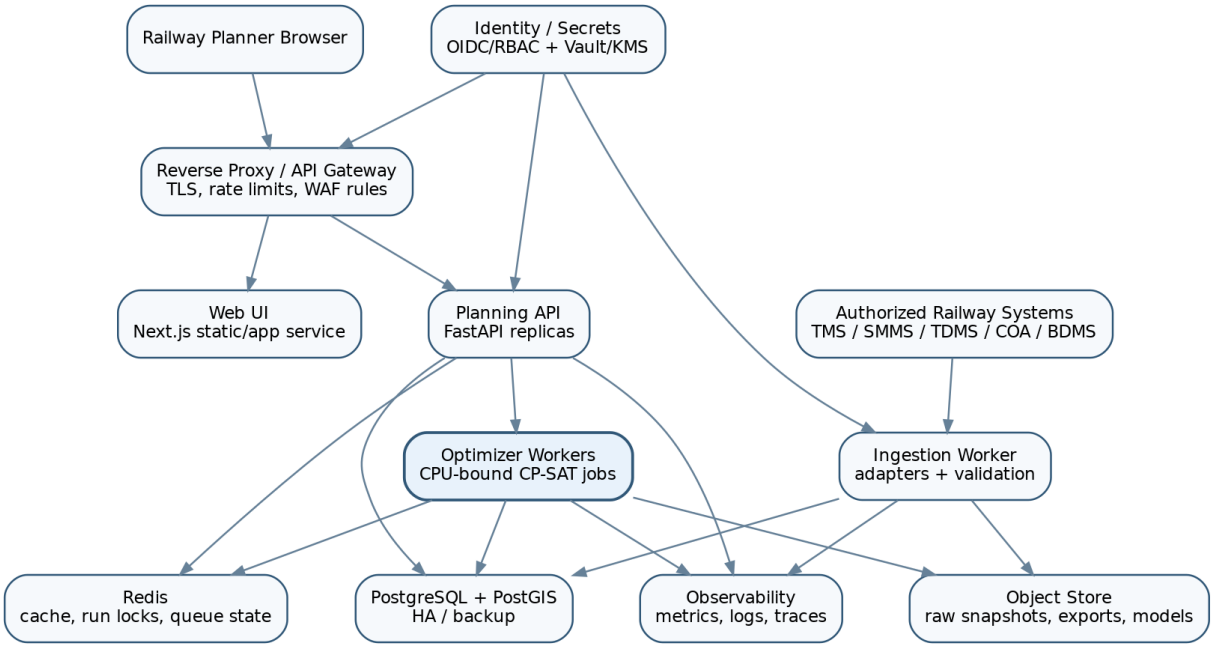
B. Planner override and re-optimization

- 1. Planner locks Block B at a chosen time and rejects one proposed block.
- 2. API validates permissions, plan version and domain validity; stores overrides with justification.
- 3. Orchestrator clones the scenario into a new run referencing prior plan as warm start.
- 4. Locked assignments become fixed hard constraints; rejected options become forbidden constraints.
- 5. Solver minimizes changes to unaffected assignments through the stability penalty.
- 6. A new immutable plan version is created; original remains available for comparison/audit.

C. Emergency defect

- 1. New high-criticality job enters through adapter/manual authorized intake.
- 2. System creates a delta snapshot and identifies impacted corridor/time region.
- 3. Re-plan is scoped to affected sections where possible; accepted unaffected blocks are locked.
- 4. If no feasible placement exists, the system returns an escalation condition with the conflicting constraints rather than fabricating a schedule.

15. Deployment Topology



For SIH, all components can run in Docker Compose on one machine or VM. The logical boundaries should still match the production topology so scaling later is straightforward. A pilot deployment can use Kubernetes/OpenShift or Railway-approved container infrastructure if permitted.

Workload classes

- Web/API: stateless, horizontally scalable, modest CPU.
- Optimizer: CPU-bound and bursty; scale workers independently and limit concurrent heavy solves.
- Ingestion: scheduled/background workload; isolated source credentials.
- Database: stateful high-availability service with PITR backups.
- Object store: immutable snapshots and model/export artifacts.
- Observability: centralized logs/metrics/traces, separate retention policy.

Network segmentation

Place external-system adapters in an integration subnet/zone with outbound/inbound access restricted to approved endpoints. Planning services should not need direct credentials to TMS/SMMS/TDMS/COA. The adapter writes normalized data through a narrow internal interface, reducing blast radius.

16. Security and Access Control

Security must reflect that the system can expose operational planning information. The architecture should assume private deployment and controlled identities, even if the SIH prototype uses local authentication.

Controls

- Authentication: OIDC/SAML integration in production; short-lived tokens.
- RBAC: roles such as Viewer, Department Planner, Control Planner, Approver, Data Steward and Platform Admin.
- Least privilege: source adapters use read-only accounts wherever possible; BDMS handoff has separate restricted credentials.
- Encryption: TLS in transit; database/object-store encryption at rest; KMS/Vault-managed secrets.
- Audit: login, export, plan approval, ruleset change, model/config change and every override logged with actor/time.

- Input hardening: schema validation, file-size limits, CSV formula neutralization for exported spreadsheets, request limits.
- Network: deny-by-default service communication; database not exposed to user networks.
- Data minimization: only planning-required operational attributes are replicated; avoid unnecessary PII entirely.

Approval separation

Creating a candidate plan and approving/exporting a plan should be separate permissions. A production rollout can require two-person approval or electronic workflow if Railway policy mandates it.

17. Reliability, Failure Handling and Recovery

The system should fail safely. An unavailable model or stale feed should create an explicit degraded state, not silently produce an apparently normal plan.

Failure modes

Failure	Behavior	Recovery
One source feed stale	Mark snapshot DEGRADED; planner may be prevented from approval based on policy	Retry connector; compare source freshness SLA
ML service unavailable	Use deterministic rules / conservative duration bounds	Alert; retain fallback_used flag in plan
Solver timeout	Persist best feasible solution + bound/gap; mark quality status	Retry with longer budget or decomposed scope
No feasible solution	Return infeasibility/conflict diagnosis; never force illegal assignment	Planner changes constraints/window/resources
Worker crash	Lease expires; orchestrator requeues idempotently	New worker resumes from snapshot
Database unavailable	API becomes read-only/unavailable; no plan approval	HA failover / restore
Planner browser disconnects	Run continues server-side; UI reconnects using run_id	Poll/SSE reconnect
Bad source batch	Batch quarantined; last good snapshot remains available	Data steward corrects mapping and re-ingests

18. Observability and Auditability

The platform needs both operational observability and decision observability. The first answers whether the software is healthy; the second answers why a plan was produced.

Operational metrics

- Adapter success/failure, records ingested, quarantine count, source staleness.
- API latency/error rate and active users.
- Queue depth, optimizer worker utilization and solve duration.
- Database connections, query latency, storage growth.
- ML inference latency/fallback rate and feature missingness.

Decision metrics

- Objective components per run: unscheduled risk, closure minutes, path disruption, block count, stability penalty.
- Number of critical jobs scheduled/unscheduled and associated reason codes.
- Solver status, best objective, bound/gap, deterministic seed and OR-Tools version.

- Input snapshot hashes, ruleset/config/model versions.
- Planner overrides by type and frequency; repeated overrides can reveal missing constraints or bad model behavior.

Traceability requirement

From any displayed block, the system should be able to trace: plan -> planning run -> snapshot -> source records -> priority factors -> compatibility decisions -> constraint/rule version -> solver result -> user overrides. This is more valuable than a generic chatbot explanation.

19. Scalability and Performance Strategy

The combinatorial optimizer is the main scaling bottleneck. Do not scale it by simply adding web servers. Control model size and partition intelligently.

Performance tactics

- Window pruning: do not create assignment variables for windows that fail obvious section/time/work-type constraints.
- Compatibility precomputation: cache pair/group compatibility for a snapshot.
- Time discretization: use a planning granularity appropriate to operations; avoid one-minute slots if interval variables can model exact times.
- Decomposition: partition by corridor/conflict component and planning week, then coordinate shared resources across partitions.
- Rolling horizon: solve near-term periods at high fidelity, later periods at lower fidelity.
- Warm start: re-plans begin from the previously accepted solution.
- Bounded concurrency: separate interactive runs from scheduled batch runs and enforce per-user/per-division quotas.
- Precomputed occupancy: materialize section-wise path intervals for the horizon rather than recalculating timetable joins on every solve.

Performance targets for a prototype

Set measurable, scenario-specific targets instead of claiming enterprise scale: for example, weekly corridor scenarios should produce a feasible result within a bounded interactive budget, while monthly runs may be asynchronous and longer. Record the number of jobs, sections, windows and solver variables alongside runtime so results are credible.

20. MLOps and Configuration Governance

The optimizer depends heavily on rules and weights; these deserve the same governance as ML models. Treat rulesets, objective weights, work-type duration bounds and safety margins as versioned configuration artifacts.

Governed artifacts

- ML model binary + feature schema + training dataset reference + evaluation report.
- Priority score rule weights and threshold mapping.
- Duration fallback table by work type/asset class.
- Compatibility/rule decision tables.
- Optimizer objective weights and solve-time limits.
- Section topology/conflict-group master data.
- Approval policies for degraded data or suboptimal solver status.

Promotion flow

DEV -> validated scenario benchmark -> reviewer approval -> STAGING -> controlled pilot -> PROD. A new ruleset/model should be replayed against saved historical/synthetic planning snapshots and compared with the current version before promotion.

21. API Contracts and Payload Examples

Create run

```
POST /v1/runs { "scenario_id": "scn_123", "snapshot_id": "snap_456", "horizon": "WEEKLY",
"objective_profile": "SAFETY_FIRST_V1", "time_limit_seconds": 60, "warm_start_plan_id": null } 202
Accepted { "run_id": "run_789", "state": "QUEUED" }
```

Run result

```
GET /v1/runs/run_789 { "state": "COMPLETED", "solver": { "status": "FEASIBLE", "runtime_ms": 18420,
"objective": 12873, "best_bound": 12610 }, "plan_id": "plan_101", "quality_flags": [] }
```

Planner lock

```
POST /v1/plans/plan_101/locks If-Match: "version-3" { "block_id": "blk_44", "lock_type":
"TIME_AND_SECTION", "justification": "Control decision / operational requirement" } 201 Created {
"override_id": "ovr_72", "new_plan_version": 4 }
```

22. Repository and Service Structure

A clean monorepo is practical for SIH and keeps contracts synchronized.

```
rail-samanvay/ apps/ web/ # Next.js/React planner UI api/ # FastAPI public/application API services/
ingest/ # source adapters + validation optimizer/ # constraint builder + CP-SAT worker intelligence/ #
priority/duration inference packages/ domain/ # shared canonical models/enums rules/ # versioned
compatibility/safety rules contracts/ # OpenAPI / JSON schemas synthetic-data/ # transparent demo data
generator data/ sample-scenarios/ # reproducible SIH scenarios infra/ docker-compose/ k8s/ # optional
pilot manifests migrations/ tests/ unit/ integration/ optimizer/ # feasibility + constraint tests
scenarios/ # golden before/after benchmarks docs/ architecture/ data-dictionary/ rules-catalog/
```

Testing architecture

- Unit-test each constraint independently with tiny counterexamples.
- Property tests: produced plans never violate no-overlap/resource/candidate-window constraints.
- Golden scenario tests: saved input snapshot must reproduce the same or no-worse objective under a fixed solver version/seed.
- Adapter contract tests against anonymized/synthetic source fixtures.
- API integration tests for stale-version conflicts and RBAC.
- Load tests for run queue and plan retrieval, not just optimizer runtime.

23. Technology Stack

Area	Recommended	Reason
Frontend	React / Next.js, TypeScript, Gantt/timeline component, map optional	Fast interactive planner UI and type-safe API client
Backend API	Python FastAPI + Pydantic	Matches optimization/ML ecosystem and provides OpenAPI contracts
Optimization	Google OR-Tools CP-SAT	Strong interval/resource scheduling primitives
Data/ML	Python, pandas/polars, scikit-learn/XGBoost only if justified	Interpretable tabular prediction; easy fallback rules

Area	Recommended	Reason
Database	PostgreSQL + PostGIS	Transactional canonical store + spatial corridor support
Cache/run coordination	Redis (optional for SIH, useful in pilot)	Progress/cache/worker coordination
Async jobs	Celery/RQ/Arq or a lightweight worker queue	Long-running solver isolation from request threads
Object storage	MinIO in prototype; S3-compatible in pilot	Immutable source/snapshot/model artifacts
Observability	OpenTelemetry + Prometheus/Grafana + structured logs	Unified traces/metrics/log correlation
Deployment	Docker Compose for demo; Kubernetes/OpenShift optional for pilot	Reproducibility then controlled horizontal scaling
Auth	Prototype local RBAC; production OIDC/SAML	Integrates with enterprise identity

24. Prototype-to-Production Path

Stage	Architecture level	What changes
SIH demo	Single host Docker Compose; FileAdapters; PostgreSQL; one optimizer worker; synthetic maintenance data	Focus on correct domain model, constraints, reproducible scenarios and UI
Railway mentor/pilot validation	Secure staging; authorized extracts; real section master; validated rules; multiple workers	Replace synthetic assumptions, calibrate durations/priority, collect override feedback
Controlled divisional pilot	HA DB, identity integration, source adapters, backups, monitoring, approval workflow	Operational reliability and governance become mandatory
Scaled deployment	Partitioned optimizer workers, rolling horizon, multi-division config, mature observability	Tune model size and data ownership; preserve division-level isolation where required

25. Architecture Assumptions and Open Railway-Domain Validations

The architecture can be built and demonstrated now, but several operational details must be validated before claiming real-world executability. Keeping them configurable is an architectural requirement.

Items to validate with authorized Railway domain experts

- Exact definitions of a legal maintenance block/disconnection for Engineering, S&T; and TRD work types.
- Minimum/maximum block durations, setup/restoration margins and permissible simultaneous activities.
- How COA, TMS, SMMS, TDMS and BDMS expose data/interfaces in the target environment and which system is authoritative for each field.
- Rules for passenger/freight priority and whether any path impact is a hard prohibition vs a weighted penalty.
- Crew/machine resource availability granularity and whether it is already maintained in a source system.
- Granularity of section/conflict/isolation zones used by real planners.
- Approval flow and required roles for exporting/sanctioning a recommended block plan.
- Data retention, hosting, network-security and audit requirements for a Railway deployment.

Architecture summary

The recommended system is not an LLM-led scheduler. It is a versioned data-integration and constraint-optimization platform: adapters -> canonical snapshot -> interpretable priority/duration estimates -> compatibility rules -> CP-SAT model -> explainable candidate plan -> human locks/approval -> re-optimization and audit. This structure directly supports the weekly/monthly, multi-department and asset-availability requirements of SIH26027.

Problem-statement grounding: SIH26027, Ministry of Railways, Smart India Hackathon 2026. Public mirrors of the statement identify TMS, SMMS, TDMS, COA/Control Office, Train Time Table, goods-train forecast, BDMS requests, and weekly/monthly optimized planning as the expected scope. Exact internal interfaces and operating rules are intentionally treated as validation items rather than invented facts.

Prepared 27 August 2026.